

Ejecta and neutrino analysis routines for AthenaK

This document summarizes the routines and equations used to compute the ejecta properties and the neutrino luminosities/average energies from the spherical-surface output of an AthenaK simulation. Both pipelines share the same structure: dump a set of fields onto an extraction sphere, process every snapshot in parallel, and accumulate the per-snapshot results in time.

Ejecta analysis

Spherical output

The ejecta analysis operates on data dumped onto AthenaK spheres. One output block is needed per variable; a block looks as follows:

```
<output1>
file_type   = sph
variable    = mhd_w_bcc
dt          = 10.0
radius      = 400.0
ntheta      = 256
```

Six variables are required at each output time and radius: the primitive MHD variables (`mhd_w_bcc`), the ADM three-metric γ_{ij} (`adm`), the lapse α (`z4c_alpha`), and the three shift components β^i (`z4c_betax`, `z4c_betay`, `z4c_betaz`). The blocks must all use the same `radius`, since the pipeline matches files by the radius string in the file name. Useful radii are 300.0 and 400.0 (code units, $M_\odot$). Spherical output is storage intensive, so `dt` should be chosen as a compromise: small enough to resolve the ejecta rate in time, large enough to keep the data volume manageable.

Pipeline

The pipeline lives under https://github.com/yiqu0714/AthenaK_BNS_visualization/tree/main/sph_data_analysis. The script `analyze_ejecta.py` does the bulk of the work. Its inputs are:

- the batchtools (<https://github.com/computationalrelativity/batchtools/tree/master>) segment directories holding the sph output (`SPH_DIR_LIST`);
- the path to the EOS table in `.h5` format (`EOS_TABLE_PATH`), which can be produced with PyCompOSE (<https://github.com/computationalrelativity/PyCompOSE>);
- the extraction radius of the spherical output (`RADIUS`);
- the density floor used in the simulation (`DFLOOR`);
- the merger time in geometric units (`T_MERGER_MSUN`) together with the post-merger window (`T_POST_MS`), which set the cutoff time `T_STOP` of the histograms.

These are set in the configuration block at the top of the script, but in practice it is easier to drive everything through `run_analysis.sh`: it reads the simulation path and the EOS table from `config.ini`, auto-discovers all `output-*/sph` segment directories, obtains the merger time from the GW (2,2) mode via `merger_time.py`, overrides the module-level settings accordingly, and then calls `analyze_ejecta.main()`. The ejecta and neutrino analyses share no state and are run concurrently by the wrapper.

Workflow

EOS SETUP The script loads the 3D `.h5` equation of state and builds `RegularGridInterpolator` objects on the table axes ($\ln n_b, Y_e, \ln T$), where n_b is the baryon number density [fm^{-3}], Y_e the electron fraction and T the temperature [MeV]. The two interpolated quantities are the tabulated CompOSE grids

$$Q_7 = \frac{e}{n_b m_n} - 1, \quad (1)$$

$$\frac{Q_1}{m_n} = \frac{p}{n_b m_n} \quad (2)$$

i.e. the specific internal energy and the pressure normalized by the rest-mass energy density. Here e is the energy density, p the pressure and m_n the neutron mass. From the same grids the minimum specific enthalpy over the whole table is computed,

$$h_{\min} = \min \left(1 + Q_7 + \frac{Q_1}{m_n} \right) = \min \left(\frac{e}{n_b m_n} + \frac{p}{n_b m_n} \right), \quad (3)$$

which is the reference value for the Bernoulli unbound criterion.

LOADING A SNAPSHOT Each of the six sph files of a snapshot is read with the `SphericalData` class of plot-tools (<https://github.com/jfields7/plot-tools>). From `mhd.w.bcc` we extract the rest-mass density ρ (`dens`), the electron fraction Y_e (`s_00`), the temperature T (`temperature`) and the velocity components $\tilde{u}^x, \tilde{u}^y, \tilde{u}^z$ (`velx`, `vely`, `velz`), which are the Valencia-frame spatial four-velocity components. From `adm` we extract the metric components $g_{xx}, g_{xy}, g_{xz}, g_{yy}, g_{yz}, g_{zz}$, and from the gauge files the lapse α and the shift $\beta^x, \beta^y, \beta^z$.

The sph output repeats points on the seam and at the pole, so every field is multiplied by a duplication-correction factor. The factor is determined from the `z4c.alpha` field: if the values at $\phi = \phi_{\min}$ exceed one, that seam is duplicated and gets weight 1/2; if the values at $\theta = \theta_{\min}$ exceed one, the pole ring is duplicated four times and gets weight 1/4. Elsewhere the factor is 1.

THERMODYNAMICS The baryon number density is obtained from ρ by a unit conversion, $n_b = \rho_{\text{cgs}} \cdot 10^{-39}/m_n [\text{fm}^{-3}]$. The triples $(\ln n_b, Y_e, \ln T)$ are clipped to the table bounds and inserted into the two interpolators, yielding $e_{\text{intp}} \equiv Q_7$ and $(p/\rho)_{\text{intp}} \equiv Q_1/m_n$ at every surface point. The specific enthalpy then follows as

$$h = 1 + e_{\text{intp}} + (p/\rho)_{\text{intp}}. \quad (4)$$

KINEMATICS The Lorentz factor is

$$W = \sqrt{1 + g_{xx}\tilde{u}_x^2 + g_{yy}\tilde{u}_y^2 + g_{zz}\tilde{u}_z^2 + 2g_{xy}\tilde{u}_x\tilde{u}_y + 2g_{xz}\tilde{u}_x\tilde{u}_z + 2g_{yz}\tilde{u}_y\tilde{u}_z}, \quad (5)$$

and gives the time component of the four-velocity

$$u^t = \frac{W}{\alpha}. \quad (6)$$

The spatial four-velocity components in the coordinate frame and their radial projection are

$$u^x = \tilde{u}^x - \beta^x u^t, \quad (7)$$

$$u^y = \tilde{u}^y - \beta^y u^t, \quad (8)$$

$$u^z = \tilde{u}^z - \beta^z u^t, \quad (9)$$

$$u_r = u^x \sin \theta \cos \phi + u^y \sin \theta \sin \phi + u^z \cos \theta. \quad (10)$$

The covariant time component, which decides whether a fluid element is bound, is

$$u_t = -\alpha W + g_{xx}\beta^x\tilde{u}^x + g_{yy}\beta^y\tilde{u}^y + g_{zz}\beta^z\tilde{u}^z \quad (11)$$

$$+ g_{xy}(\beta^x\tilde{u}^y + \beta^y\tilde{u}^x) + g_{xz}(\beta^x\tilde{u}^z + \beta^z\tilde{u}^x) + g_{yz}(\beta^y\tilde{u}^z + \beta^z\tilde{u}^y). \quad (12)$$

MASKS A point contributes to the outflow if it lies above the density floor and moves outwards, $\rho > \rho_{\text{floor}}$ and $u_r > 0$. Within that outflow mask, the unbound points are selected by:

- Geodesic criterion: $u_t < -1$
- Bernoulli criterion: $h u_t < -h_{\min}$

SURFACE INTEGRAND With the determinant of the three-metric

$$\gamma = g_{xx}g_{yy}g_{zz} + 2g_{xy}g_{xz}g_{yz} - g_{xx}g_{yz}^2 - g_{yy}g_{xz}^2 - g_{zz}g_{xy}^2, \quad (13)$$

the base integrand of the mass flux through the sphere reads

$$b = \rho u_r \sqrt{\gamma} \alpha r^2 \sin \theta. \quad (14)$$

Multiplying by the per-cell Riemann weights R_w (the products $\Delta\theta \Delta\phi$ from the midpoint rule, with the pole cells divided by N_ϕ so the singular axis is not counted N_ϕ times) gives the cell rate $c_r = b R_w$. Applying the geodesic and the Bernoulli mask to c_r defines the corresponding flux rates f_r , and the ejecta rate of each criterion is their sum over the surface,

$$\dot{M}_{\text{ej}} = \sum f_r. \quad (15)$$

The velocity at infinity is evaluated from the same quantities that enter the masks,

$$v_\infty^{\text{geo}} = \sqrt{1 - u_t^{-2}} \quad \text{where } u_t < -1, \quad (16)$$

$$v_\infty^{\text{Ber}} = \sqrt{1 - (hu_t)^{-2}} \quad \text{where } hu_t < -h_{\text{min}}, \quad (17)$$

and is set to zero elsewhere. The mass-averaged velocity at infinity is the flux-weighted mean over the surface,

$$\langle v_\infty \rangle = \frac{\sum v_\infty f_r}{\sum f_r}, \quad (18)$$

and the mass-averaged electron fraction $\langle Y_e \rangle$ is computed in exactly the same way with v_∞ replaced by Y_e . Both are evaluated separately for the geodesic and the Bernoulli mask. In addition, an unbound-criterion-independent rate `Mej_rate` is accumulated over the plain outflow mask. This workflow is applied to every sph snapshot.

Post-Processing

Parallel workers apply the workflow above to each snapshot, and the results are sorted in time afterwards. Snapshots earlier than `T_STOP` ($= t_{\text{merger}} + \text{T_POST_MS}$) additionally contribute a 2D histogram in (v_∞, θ) and a 1D histogram in Y_e , both weighted by the flux rate; the time series of the rates themselves use all snapshots. The per-snapshot histograms are accumulated in time with a midpoint Riemann sum, which yields the ejected mass per bin $dM_{\text{ej}}(v_\infty, \theta)$ [$M_\odot$]. Its marginals reproduce both the v_∞ distribution (sum over θ) and the angular distribution $dM_{\text{ej}}/d\theta$ (sum over v_∞ , divided by the bin width $\Delta\theta$), so the two are not computed separately. The total ejected mass is obtained by integrating the ejecta rate in time with Simpson's rule,

$$M_{\text{ej}} = \int \dot{M}_{\text{ej}} dt. \quad (19)$$

The rates for each criterion, the mass-averaged $\langle v_\infty \rangle$ and $\langle Y_e \rangle$ time series, the v_∞ , θ and Y_e marginals, and the full 2D arrays are written to `.txt`/`.npy` files in `OUTPUT_DIR` for further analysis. Passing `--plot` to `run_analysis.sh` visualizes them with `plot_analysis.py`; the result looks like Fig. ??.

Neutrino analysis

Spherical output

The neutrino analysis likewise operates on data dumped onto AthenaK spheres, with one output block per variable:

```
<output2>
file_type = sph
variable  = rad_m1_N
dt        = 10.0
radius    = 400.0
ntheta    = 256
```

Eight variables are required at each output time and radius: the Eulerian energy-flux covector per species F_i (`rad_m1_F`), the Eulerian energy density per species E (`rad_m1_E`), the number density per species N (`rad_m1_N`), the ADM three-metric γ_{ij} (`adm`), the lapse α (`z4c_alpha`) and the three shift components β^i (`z4c_betax`, `z4c_betay`, `z4c_betaz`). Useful radii are again 300.0 and 400.0, and the same trade-off applies when choosing `dt`.

Pipeline

The pipeline lives under https://github.com/yiqiu0714/AthenaK_BNS_visualization/tree/main/sph_data_analysis, where `analyze_neutrinos.py` does the bulk of the computation. As input it takes the batchtools (<https://github.com/computationalrelativity/batchtools/tree/master>) segment directories containing the sph output (`SPH_DIR_LIST`) and the extraction radius (`RADIUS`). As for the ejecta, both can be left to `run_analysis.sh`, which sets them from `config.ini` and can also produce the plots.

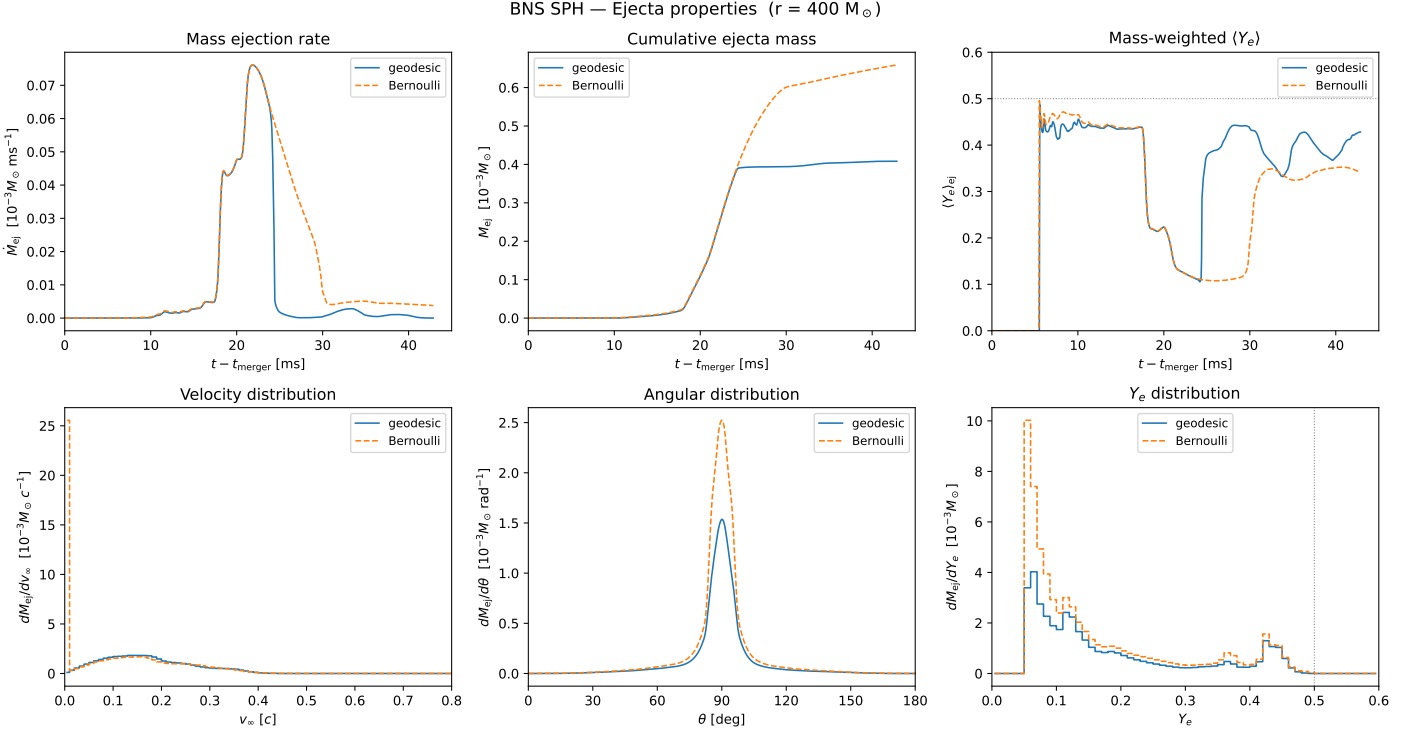


Figure 1: Example of the ejecta analysis described above.

Workflow

The supported species are ν_e (**nue**), $\bar{\nu}_e$ (**nua**), ν_x (**nux**) and $\bar{\nu}_x$ (**anux**), where ν_x stands for the heavy-lepton neutrinos; they follow AthenaK's M1 species ordering $s = 0 \dots 3$.

FILE MATCHING The pipeline first indexes the **rad.m1.F** files and builds a time list for the **adm** and gauge files by reading only their VTK header. This is necessary because the M1 and the metric output may use different counter series (e.g. after a restart) even though they cover the same simulation times. Taking the times of the neutrino fluxes F as the baseline, the nearest **adm**, **z4c.alpha** and **z4c.beta*** file in time is selected for each snapshot; snapshots without a match inside the tolerance are skipped and reported. The data are then loaded with the **SphericalData** class of plot-tools (<https://github.com/jfields7/plot-tools>), and separate workers process each snapshot to compute the neutrino luminosities and average energies.

ANGULAR WEIGHTS AND DUPLICATES From the flux file we take the time, the radius r , the angles θ , ϕ , and the native AthenaK surface weights $w = r^2 d\Omega$ (Gauss-Legendre in $\mu = \cos \theta$, uniform in ϕ), which are used directly for the surface integrals. The θ bin edges for the angular diagnostics are built from the first snapshot. As in the ejecta analysis, the duplication correction is derived from **z4c.alpha** and applied to all eight output variables.

RADIAL PROJECTION For each species we extract the flux components F_x , F_y , F_z , the energy density E and the number density N , and raise the index on the flux with the inverse three-metric:

$$F^x = \gamma^{xx} F_x + \gamma^{xy} F_y + \gamma^{xz} F_z \quad (20)$$

$$F^y = \gamma^{xy} F_x + \gamma^{yy} F_y + \gamma^{yz} F_z \quad (21)$$

$$F^z = \gamma^{xz} F_x + \gamma^{yz} F_y + \gamma^{zz} F_z \quad (22)$$

The inverse three-metric is computed component-wise from the determinant γ :

$$\gamma = g_{xx}g_{yy}g_{zz} + 2g_{xy}g_{xz}g_{yz} - g_{xx}g_{yz}^2 - g_{yy}g_{xz}^2 - g_{zz}g_{xy}^2 \quad (23)$$

$$\gamma^{xx} = (g_{yy}g_{zz} - g_{yz}^2) / \gamma \quad (24)$$

$$\gamma^{yy} = (g_{xx}g_{zz} - g_{xz}^2) / \gamma \quad (25)$$

$$\gamma^{zz} = (g_{xx}g_{yy} - g_{xy}^2) / \gamma \quad (26)$$

$$\gamma^{xy} = (g_{xz}g_{yz} - g_{xy}g_{zz}) / \gamma \quad (27)$$

$$\gamma^{xz} = (g_{xy}g_{yz} - g_{xz}g_{yy}) / \gamma \quad (28)$$

$$\gamma^{yz} = (g_{xy}g_{xz} - g_{yz}g_{xx}) / \gamma \quad (29)$$

Following THC, the radial direction is normalized with the spatial block of the inverse four-metric, $g_4^{ij} = \gamma^{ij} - \beta^i \beta^j / \alpha^2$:

$$g_4^{xx} = \gamma^{xx} - \beta^x \beta^x / \alpha^2 \quad g_4^{xy} = \gamma^{xy} - \beta^x \beta^y / \alpha^2 \quad (30)$$

$$g_4^{yy} = \gamma^{yy} - \beta^y \beta^y / \alpha^2 \quad g_4^{xz} = \gamma^{xz} - \beta^x \beta^z / \alpha^2 \quad (31)$$

$$g_4^{zz} = \gamma^{zz} - \beta^z \beta^z / \alpha^2 \quad g_4^{yz} = \gamma^{yz} - \beta^y \beta^z / \alpha^2 \quad (32)$$

The radial vector is constructed from the angles (θ, ϕ) and the radius of the sphere,

$$r_x = r \sin \theta \cos \phi, \quad r_y = r \sin \theta \sin \phi, \quad r_z = r \cos \theta, \quad (33)$$

and its metric-normalized magnitude is

$$|r| = \sqrt{g_4^{xx}r_x^2 + g_4^{yy}r_y^2 + g_4^{zz}r_z^2 + 2g_4^{xy}r_xr_y + 2g_4^{xz}r_xr_z + 2g_4^{yz}r_yr_z}. \quad (34)$$

The flux projected along the radial direction is then

$$F^r = \frac{r_x F^x + r_y F^y + r_z F^z}{|r|}, \quad (35)$$

and the same projection applied to the shift gives

$$\beta^r = \frac{r_x \beta^x + r_y \beta^y + r_z \beta^z}{|r|}. \quad (36)$$

LUMINOSITY AND AVERAGE ENERGY The radial energy flux of each species is

$$F_{\text{rad}} = \alpha F^r - \beta^r E. \quad (37)$$

Note that E and F_i are already densitized M1 variables, so no additional $\sqrt{\gamma}$ enters the surface integral. The energy luminosity in code units is

$$L_{\text{code}} = \sum F_{\text{rad}} w, \quad (38)$$

which is converted to ergs^{-1} for each species. For the average energy we need the number luminosity. The M1 number current f_ν^a is not part of the sph output, so the number flux is reconstructed by assuming that the number and energy currents are advected with the same radial velocity, $F_N^r \approx F_{\text{rad}} N/E$ (exact in the free-streaming limit at the detector sphere). With N converted from fm^{-3} to code units $M_\odot^{-3}$,

$$\dot{N}_{\text{code}} = \sum \frac{F_{\text{rad}} N_{\text{code}}}{E} w, \quad (39)$$

where the sum runs over the points with $E > 0$. The flux-weighted average neutrino energy per species is the ratio of the two,

$$\langle \epsilon_\nu \rangle = \frac{L_{\text{code}}}{\dot{N}_{\text{code}}}, \quad (40)$$

converted to MeV. Finally, the angular distribution of the luminosity $dL_\nu/d\theta$ is obtained by histogramming the outgoing part of the radial flux ($F_{\text{rad}} > 0$) in θ with weights $F_{\text{rad}} w$ and dividing by the bin width $\Delta\theta$.

BNS SPH — Neutrino emission (r = 400 M_⊙)

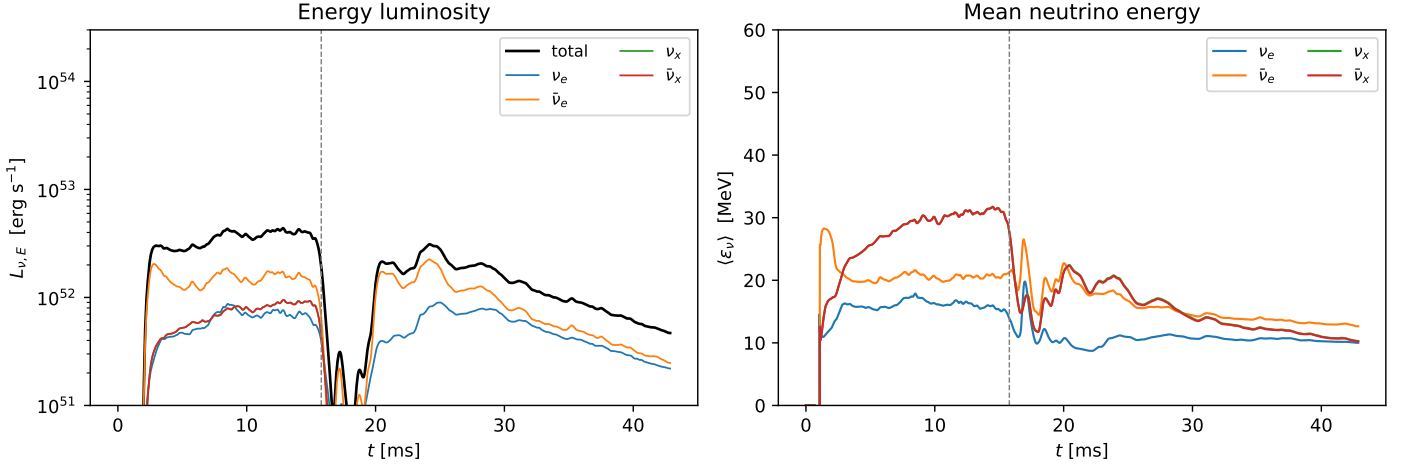


Figure 2: Example of the neutrino analysis described above.

Post-Processing

The above workflow is applied to all spherical output snapshots and the results are sorted in time. We store the energy luminosity L_ν and the average energy $\langle \epsilon_\nu \rangle$ per species as time series, as well as the total luminosity summed over all four species, $L_{\text{total}} = \sum_\nu L_\nu$. The angular profiles $dL_\nu/d\theta$ are integrated over time with a midpoint Riemann sum and converted to erg, which gives the time-integrated radiated energy per angle $dE_\nu/d\theta$ [erg rad⁻¹] for each species. All of these are written to `.txt` files in `OUTPUT_DIR` for easy processing: `Lnu_E_{sp}.txt`, `Lnu_E.total.txt`, `Eav_{sp}.txt` and `dEnu_dtheta_{sp}.txt`. With the `--plot` flag of `run_analysis.sh` one can plot the neutrino luminosity and the average neutrino energy for each species; the results look like Fig. ??.